

Particle Swarm Optimization for IRS system

July 2026

1 Initialization

Before executing the Particle Swarm Optimization (PSO) algorithm, all communication system parameters are initialized according to the IRS-assisted wireless communication model. These parameters define the physical environment, transmission power, channel attenuation, and practical hardware characteristics of the Intelligent Reflecting Surface (IRS).

1.1 System Parameters

The Access Point (AP) is equipped with

$$M = 2$$

transmit antennas, while the user is assumed to have a single receive antenna. The transmit power of the AP is

$$P_T = 36 \text{ dBm},$$

which is converted into watts for numerical computation using

$$P_T = 10^{\frac{P_{T,\text{dBm}} - 30}{10}}.$$

The receiver noise power is

$$\sigma^2 = -94 \text{ dBm},$$

and is similarly converted into watts by

$$\sigma^2 = 10^{\frac{\sigma_{\text{dBm}}^2 - 30}{10}}.$$

Unlike the original phase-shift optimization problem, the proposed approach directly optimizes the circuit parameters of each IRS reflecting element. The operating frequency is

$$f = 2.4 \text{ GHz},$$

with the corresponding angular frequency

$$\omega = 2\pi f.$$

The free-space impedance is fixed as

$$Z_0 = 377 \, \Omega.$$

For each reflecting element, the optimization variables are the inductances, capacitance, and resistance,

$$L_{1,n}, \quad L_{2,n}, \quad C_n, \quad R_n,$$

whose search ranges are defined as

$$\begin{aligned} 1 \, \text{nH} &\leq L_{1,n} \leq 6 \, \text{nH}, \\ 0.2 \, \text{nH} &\leq L_{2,n} \leq 3 \, \text{nH}, \\ 0.1 \, \text{pF} &\leq C_n \leq 10 \, \text{pF}, \\ 0.1 \, \Omega &\leq R_n \leq 20 \, \Omega. \end{aligned}$$

For every candidate solution generated by PSO, the equivalent impedance of each IRS element is first computed as

$$Z_n = \frac{j\omega L_{1,n} \left(j\omega L_{2,n} + \frac{1}{j\omega C_n} + R_n \right)}{j\omega L_{1,n} + \left(j\omega L_{2,n} + \frac{1}{j\omega C_n} + R_n \right)},$$

and the corresponding reflection coefficient is obtained by

$$v_n = \frac{Z_n - Z_0}{Z_n + Z_0}.$$

The reflection coefficients of all IRS elements form the reflection vector

$$\mathbf{v} = [v_1, v_2, \dots, v_N]^T,$$

which is then used to evaluate the achievable rate

$$R = \log_2 \left(1 + \frac{P_T \|\mathbf{v}^H \mathbf{\Phi} + \mathbf{h}_d^H\|^2}{\sigma^2} \right),$$

where P_T denotes the transmit power, σ^2 is the average Gaussian noise power at the receiver, $\mathbf{\Phi}$ is the cascaded AP-IRS channel, and $\mathbf{h}_d$ is the direct AP-user channel.

1.2 Path Loss Parameters

Large-scale fading is modeled using the standard distance-dependent path loss model

$$PL(d) = PL_{1m}d^{-\alpha},$$

where

$$PL_{1m} = 10^{-40/10} = 10^{-4}$$

is the reference path loss at a distance of one meter.

Different communication links experience different propagation environments; therefore, different path loss exponents are adopted:

- $\alpha_{AI} = 2.2$ for the AP–IRS link,
- $\alpha_{IU} = 2.8$ for the IRS–User link,
- $\alpha_{AU} = 3.8$ for the direct AP–User link.

A smaller path loss exponent indicates a stronger propagation channel. Consequently, the AP–IRS link experiences the least attenuation, while the direct AP–User link undergoes the most severe attenuation. These path loss coefficients are used to determine the large-scale attenuation of each communication link. The resulting path losses are then combined with Rayleigh small-scale fading to generate the AP–IRS, IRS–User, and AP–User channel coefficients used throughout the simulations.

1.3 Channel Initialization

After computing the large-scale path loss of each communication link, the wireless channels are generated according to the Rayleigh fading model. The channel between the Access Point (AP) and the IRS is denoted by the matrix

$$\mathbf{G} \in \mathbb{C}^{N \times M},$$

where N is the number of IRS reflecting elements and M is the number of transmit antennas at the AP. Each element of $\mathbf{G}$ represents the complex channel coefficient from one AP antenna to one IRS element. The channel is generated as

$$\mathbf{G} = \sqrt{\frac{PL_{AI}}{2}} (\mathbf{X} + j\mathbf{Y}),$$

where

$$\mathbf{X}, \mathbf{Y} \sim \mathcal{N}(0, 1)$$

are independent real Gaussian random matrices. Consequently,

$$\mathbf{G} \sim \mathcal{CN}(\mathbf{0}, PL_{AI}\mathbf{I}),$$

which models a Rayleigh fading channel with average power determined by the path loss PL_{AI} . The scaling factor

$$\sqrt{\frac{PL_{AI}}{2}}$$

ensures that the generated complex Gaussian channel has the desired average power after incorporating large-scale fading. Similarly, the IRS–User channel vector

$$\mathbf{h}_r \in \mathbb{C}^{N \times 1}$$

and the direct AP–User channel vector

$$\mathbf{h}_d \in \mathbb{C}^{M \times 1}$$

are generated using the same procedure with their corresponding path losses PL_{IU} and PL_{AU} , respectively. Finally, the equivalent cascaded channel is pre-computed as

$$\Phi = \text{diag}(\mathbf{h}_r^H) \mathbf{G},$$

where $\text{diag}(\mathbf{h}_r^H)$ is a diagonal matrix whose diagonal entries are the conjugate channel coefficients between the IRS and the user. Precomputing the matrix Φ avoids repeatedly evaluating the product

$$\text{diag}(\mathbf{h}_r^H) \mathbf{G}$$

during every fitness evaluation, thereby reducing the computational complexity of the PSO optimization.

1.4 Python Implementation

The following code initializes the communication system parameters and generates the wireless channels used throughout the PSO optimization process.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # --- SYSTEM PARAMETERS (BASED ON THE REFERENCE PAPER) ---
5 M = 2 # Number of transmit antennas at the AP
6
7 P_T_dBm = 36 # Transmit power at the AP (dBm)
8 P_T = 10**((P_T_dBm - 30) / 10) # Convert to watts
9
10 sigma2_dBm = -94
11 sigma2 = 10**((sigma2_dBm - 30) / 10) # Noise power (watts)
12
13 # Path loss parameters (reference path loss at 1 meter: -40
14 # dB)
15 PL_1m = 10**(-40 / 10)
16 alpha_AI = 2.2 # AP-to-IRS path loss exponent

```

```

16 alpha_IU = 2.8    # IRS-to-User path loss exponent
17 alpha_AU = 3.8    # AP-to-User path loss exponent
18
19 # Geometric locations
20 x_AP, y_AP = 0, 0
21 x_IRS, y_IRS = 500, 0
22
23 # =====
24 # IRS Reflection Model
25 # =====
26
27 # Operating frequency and circuit parameters
28 f = 2.4e9          # Operating frequency (Hz)
29 omega = 2 * np.pi * f      # Angular frequency (rad/s)
30 Z0 = 377           # Free-space impedance (Ohms)
31
32 # Search ranges of circuit components
33 L1_MIN = 1e-9
34 L1_MAX = 6e-9
35
36 L2_MIN = 0.2e-9
37 L2_MAX = 3e-9
38
39 C_MIN = 0.1e-12
40 C_MAX = 10e-12
41
42 R_MIN = 0.1
43 R_MAX = 20
44 def get_path_loss(d, alpha):
45     return PL_1m * (d ** (-alpha))
46
47 def generate_channels(N, d_user_x):
48     y_user = 2
49     d_AI = 500
50     d_AU = np.sqrt(d_user_x**2 + y_user**2)
51     d_IU = np.sqrt((500 - d_user_x)**2 + y_user**2)
52
53     PL_AI = get_path_loss(d_AI, alpha_AI)
54     PL_IU = get_path_loss(d_IU, alpha_IU)
55     PL_AU = get_path_loss(d_AU, alpha_AU)
56
57     G = np.sqrt(PL_AI / 2) * (np.random.randn(N, M) + 1j *
58                               np.random.randn(N, M))
59     h_r = np.sqrt(PL_IU / 2) * (np.random.randn(N, 1) + 1j *
60                               np.random.randn(N, 1))
61     h_d = np.sqrt(PL_AU / 2) * (np.random.randn(M, 1) + 1j *
62                               np.random.randn(M, 1))
63
64     Phi = np.diag(h_r.conj().flatten()) @ G
65     return Phi, h_d

```

```

63
64 def compute_reflection(x):
65     """
66     x have form that:
67     [L1_1,L2_1,C1,R1,
68      L1_2,L2_2,C2,R2,
69      ...
70     ]
71     """
72     num = len(x) // 4
73
74     L1 = x[0::4]
75     L2 = x[1::4]
76     C = x[2::4]
77     R = x[3::4]
78
79     j = 1j
80
81     Z = (j*omega*L1 *
82          (j*omega*L2 + 1/(j*omega*C) + R)) / \
83          (j*omega*L1 +
84           (j*omega*L2 + 1/(j*omega*C) + R))
85
86     v = (Z - Z0)/(Z + Z0)
87
88     return v

```

Listing 1: System initialization and channel generation

2 Optimization Algorithm

This section presents the optimization procedure adopted to maximize the achievable rate of the IRS-assisted communication system. Instead of employing the Alternating Optimization (AO) algorithm proposed in the original paper, this implementation utilizes an Adaptive Particle Swarm Optimization (Adaptive PSO) algorithm enhanced with the Clerc–Kennedy constriction factor and the Elitist Learning Strategy (ELS).

Unlike the original formulation, which directly optimizes the IRS reflection phase shifts, the proposed approach performs optimization at the circuit level. The optimization variable is defined as

$$\mathbf{x} = [L_{1,1}, L_{2,1}, C_1, R_1, L_{1,2}, L_{2,2}, C_2, R_2, \dots, L_{1,N}, L_{2,N}, C_N, R_N]^T,$$

where each group $(L_{1,n}, L_{2,n}, C_n, R_n)$ represents the equivalent circuit parameters of the n -th IRS reflecting element.

For every candidate solution generated by the PSO algorithm, the equivalent impedance of each IRS element is first computed using the circuit model.

The corresponding reflection coefficients are then obtained to construct the IRS reflection vector

$$\mathbf{v} = [v_1, v_2, \dots, v_N]^T.$$

The reflection vector is subsequently incorporated into the achievable rate expression to evaluate the fitness of the candidate solution. The particle yielding the highest achievable rate is regarded as the current global best solution and guides the swarm toward the optimal circuit parameter configuration.

2.1 Fitness Computation

The objective of the optimization is to maximize the achievable rate of the IRS-assisted communication system. For each particle generated by the PSO algorithm, the corresponding IRS phase shift vector is substituted into the channel model to compute the equivalent communication channel

$$\mathbf{h}_{\text{eff}} = \mathbf{v}^H \Phi + \mathbf{h}_d^H,$$

where $\mathbf{v}^H \Phi$ represents the reflected transmission path through the IRS, while $\mathbf{h}_d^H$ denotes the direct transmission path between the AP and the user. Therefore, $\mathbf{h}_{\text{eff}}$ combines the contributions of both communication links into a single equivalent channel.

Assuming the AP employs Maximum Ratio Transmission (MRT), the achievable rate is computed as

$$R_s = \log_2 \left(1 + \frac{P_T \|\mathbf{h}_{\text{eff}}\|^2}{\sigma^2} \right),$$

where P_T is the transmit power of the AP, σ^2 is the receiver noise power, and $\|\mathbf{h}_{\text{eff}}\|^2$ denotes the effective channel gain. A larger effective channel gain results in a higher signal-to-noise ratio (SNR), thereby increasing the achievable rate. Consequently, the value of R_s is directly adopted as the fitness value of each particle, and the optimization process aims to maximize this quantity.

The corresponding implementation is shown below.

```

1 def compute_reflection(x):
2     """
3     x have form:
4     [L1_1, L2_1, C1, R1,
5      L1_2, L2_2, C2, R2,
6      ...
7     ]
8     """
9     num = len(x) // 4
10
11     L1 = x[0::4]
12     L2 = x[1::4]
13     C  = x[2::4]
```

```

14     R = x[3::4]
15
16     j = 1j
17
18     Z = (j*omega*L1 *
19           (j*omega*L2 + 1/(j*omega*C) + R)) / \
20           (j*omega*L1 +
21           (j*omega*L2 + 1/(j*omega*C) + R))
22
23     v = (Z - Z0)/(Z + Z0)
24
25     return v
26
27 def compute_fitness(x, Phi, h_d):
28
29     v = compute_reflection(x)
30     combined_channel = v.conj().T @ Phi + h_d.conj().T
31     norm_squared = np.sum(np.abs(combined_channel) ** 2)
32
33     return np.log2(1 + (P_T * norm_squared) / sigma2)

```

Listing 2: Fitness function implementation

2.2 Adaptive PSO with Ellitist learning strategy

2.2.1 Particle and Parameter Initialization

Before the optimization process begins, the parameters of the Adaptive PSO algorithm are initialized. The cognitive and social acceleration coefficients are chosen as

$$\phi_1 = \phi_2 = 2.05,$$

which are the recommended values proposed by Clerc and Kennedy to ensure stable convergence. Their sum is

$$\phi = \phi_1 + \phi_2,$$

and the corresponding constriction factor is computed as

$$\chi = \frac{2}{\left| 2 - \phi - \sqrt{\phi^2 - 4\phi} \right|}.$$

The constriction factor scales the particle velocity during each iteration, preventing excessive oscillations and improving the convergence stability of the PSO algorithm.

The optimization variables correspond to the equivalent circuit parameters of each IRS element. Their feasible ranges are defined as

$$\begin{aligned}
1 \text{ nH} &\leq L_{1,n} \leq 6 \text{ nH}, \\
0.2 \text{ nH} &\leq L_{2,n} \leq 3 \text{ nH}, \\
0.1 \text{ pF} &\leq C_n \leq 10 \text{ pF}, \\
0.1 \Omega &\leq R_n \leq 20 \Omega.
\end{aligned}$$

The initial particle positions are generated around the nominal circuit parameter values rather than being uniformly distributed over the entire search space. Specifically, the nominal values are chosen as

$$L_1^{\text{ref}} = 2.5 \text{ nH}, \quad L_2^{\text{ref}} = 0.7 \text{ nH}, \quad C^{\text{ref}} = 1.5 \text{ pF}, \quad R^{\text{ref}} = 2 \Omega.$$

For each particle, the circuit parameters are independently sampled from Gaussian distributions centered at these nominal values,

$$\begin{aligned}
L_{1,n} &\sim \mathcal{N}(L_1^{\text{ref}}, (0.1L_1^{\text{ref}})^2), \\
L_{2,n} &\sim \mathcal{N}(L_2^{\text{ref}}, (0.1L_2^{\text{ref}})^2), \\
C_n &\sim \mathcal{N}(C^{\text{ref}}, (0.2C^{\text{ref}})^2), \\
R_n &\sim \mathcal{N}(R^{\text{ref}}, (0.2R^{\text{ref}})^2).
\end{aligned}$$

To ensure physically feasible solutions, all sampled values are clipped to their corresponding lower and upper bounds before the optimization process begins. Consequently, each row of the particle position matrix

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_p}]^T$$

represents one candidate circuit parameter vector, where N_p denotes the number of particles in the swarm.

Similarly, the initial particle velocities are randomly generated according to the search range of each circuit parameter. Specifically,

$$\begin{aligned}
V_{L_1} &\sim \mathcal{U}(-0.1(L_{1,\max} - L_{1,\min}), 0.1(L_{1,\max} - L_{1,\min})), \\
V_{L_2} &\sim \mathcal{U}(-0.1(L_{2,\max} - L_{2,\min}), 0.1(L_{2,\max} - L_{2,\min})), \\
V_C &\sim \mathcal{U}(-0.1(C_{\max} - C_{\min}), 0.1(C_{\max} - C_{\min})), \\
V_R &\sim \mathcal{U}(-0.1(R_{\max} - R_{\min}), 0.1(R_{\max} - R_{\min})).
\end{aligned}$$

This initialization provides each particle with a moderate initial moving speed while maintaining consistency with the physical scales of different circuit parameters. Consequently, particles are able to effectively explore the search space during the early iterations without producing excessively large updates.

providing each particle with an initial moving direction and speed.

After initialization, the fitness of every particle is evaluated using the fitness function introduced in Section 2.1. The initial personal best positions are set equal to the initial particle positions,

$$\mathbf{P}_{\text{best}} = \mathbf{X},$$

while the global best particle is determined as the particle with the highest achievable rate,

$$\mathbf{G}_{\text{best}} = \arg \max_i f(\mathbf{P}_{\text{best},i}),$$

where $f(\cdot)$ denotes the fitness function.

Finally, two parameters for the Elitist Learning Strategy (ELS) are initialized,

$$\sigma_{\text{max}} = 1.0, \quad \sigma_{\text{min}} = 0.1,$$

which specify the maximum and minimum standard deviations of the Gaussian perturbation applied to the global best particle. During the optimization process, the perturbation strength gradually decreases from σ_{max} to σ_{min} , allowing the algorithm to perform global exploration in the early iterations and fine local exploitation in the later stages.

The corresponding implementation is shown below.

```

1 phi1, phi2 = 2.05, 2.05
2   phi = phi1 + phi2
3   chi = 2 / np.abs(2 - phi - np.sqrt(phi**2 - 4*phi))
4
5   DIM = 4*N
6   X=np.zeros((num_particles,DIM))
7
8   # Nominal values adopted from the reference paper
9   L1_REF = 2.5e-9      # 2.5 nH
10  L2_REF = 0.7e-9      # 0.7 nH
11  C_REF  = 1.5e-12     # choose between the range of C
12  R_REF  = 2.0         # average value
13
14  for i in range(num_particles):
15
16      X[i,0::4] = np.clip(
17          np.random.normal(L1_REF, 0.1*L1_REF, N),
18          L1_MIN, L1_MAX
19      )
20
21      X[i,1::4] = np.clip(
22          np.random.normal(L2_REF, 0.1*L2_REF, N),
23          L2_MIN, L2_MAX
24      )
25
26      X[i,2::4] = np.clip(
27          np.random.normal(C_REF, 0.2*C_REF, N),
28          C_MIN, C_MAX
29      )

```

```

30         X[i,3::4] = np.clip(
31             np.random.normal(R_REF, 0.2*R_REF, N),
32             R_MIN, R_MAX
33         )
34
35
36     V = np.zeros((num_particles, DIM))
37
38     V[:,0::4] = np.random.uniform(
39         -0.1*(L1_MAX-L1_MIN),
40         0.1*(L1_MAX-L1_MIN),
41         (num_particles, N)
42     )
43
44     V[:,1::4] = np.random.uniform(
45         -0.1*(L2_MAX-L2_MIN),
46         0.1*(L2_MAX-L2_MIN),
47         (num_particles, N)
48     )
49
50     V[:,2::4] = np.random.uniform(
51         -0.1*(C_MAX-C_MIN),
52         0.1*(C_MAX-C_MIN),
53         (num_particles, N)
54     )
55
56     V[:,3::4] = np.random.uniform(
57         -0.1*(R_MAX-R_MIN),
58         0.1*(R_MAX-R_MIN),
59         (num_particles, N)
60     )
61
62     pbest = X.copy()
63     pbest_fitness = np.array([compute_fitness(X[i], Phi, h_d
64         ) for i in range(num_particles)])
65
66     gbest_idx = np.argmax(pbest_fitness)
67     gbest = pbest[gbest_idx].copy()
68     gbest_fitness = pbest_fitness[gbest_idx]
69
70     sigma_max, sigma_min = 1.0, 0.1

```

Listing 3: Initialization of Adaptive ELS-PSO

2.2.2 Iterative Optimization

After the initialization stage, the Adaptive ELS-PSO algorithm iteratively updates the particle positions until the maximum number of iterations is reached.

Unlike the standard PSO with a fixed inertia weight, this implementation

adopts a linearly decreasing inertia weight

$$w(t) = 0.9 - 0.5 \frac{t}{T_{\max}},$$

where t denotes the current iteration and $T_{\max}$ is the maximum number of iterations. A relatively large inertia weight at the beginning encourages global exploration of the search space, while a smaller inertia weight in later iterations improves local exploitation around the current optimum.

The particle velocity is updated using the Clerc–Kennedy constriction factor,

$$\mathbf{V}_i^{t+1} = \chi [w(t)\mathbf{V}_i^t + \phi_1 r_1 (\mathbf{P}_i^{\text{best}} - \mathbf{X}_i^t) + \phi_2 r_2 (\mathbf{G}^{\text{best}} - \mathbf{X}_i^t)],$$

where r_1 and r_2 are uniformly distributed random numbers in the interval $[0, 1]$, ϕ_1 and ϕ_2 are the cognitive and social acceleration coefficients, respectively, and χ is the Clerc–Kennedy constriction factor introduced in the previous subsection. The constriction factor suppresses excessive particle oscillation and improves the convergence stability of the swarm.

After updating the velocity, the particle position is updated according to

$$\mathbf{X}_i^{t+1} = \mathbf{X}_i^t + \mathbf{V}_i^{t+1}.$$

Since the optimization variables correspond to physical circuit parameters, each updated component value is clipped to its predefined search range,

$$\begin{aligned} L_{1,\min} &\leq L_{1,n} \leq L_{1,\max}, \\ L_{2,\min} &\leq L_{2,n} \leq L_{2,\max}, \\ C_{\min} &\leq C_n \leq C_{\max}, \\ R_{\min} &\leq R_n \leq R_{\max}. \end{aligned}$$

The updated circuit parameter vector is subsequently converted into the IRS reflection coefficients through the equivalent circuit model, and the achievable rate is evaluated using the fitness function described in the previous subsection. If the updated particle produces a higher fitness value than its previous personal best, the personal best is replaced accordingly. After all particles have been evaluated, the global best solution is updated by selecting the particle with the highest fitness.

To further reduce the possibility of premature convergence, an Elitist Learning Strategy (ELS) is applied after each PSO iteration. The perturbation strength is first determined by

$$\sigma(t) = \sigma_{\max} - (\sigma_{\max} - \sigma_{\min}) \frac{t}{T_{\max}},$$

which gradually decreases from $\sigma_{\max}$ to $\sigma_{\min}$ as the optimization proceeds.

A temporary solution is then generated by copying the current global best particle and randomly selecting one optimization variable for Gaussian perturbation,

$$x'_d = x_d + (x_{d,\max} - x_{d,\min})\mathcal{N}(0, \sigma),$$

where x_d denotes the selected circuit parameter, $x_{d,\min}$ and $x_{d,\max}$ are its corresponding lower and upper bounds, and $\mathcal{N}(0, \sigma)$ represents a zero-mean Gaussian random variable with standard deviation σ .

The perturbed parameter is clipped to its feasible interval before the fitness value is evaluated. If the perturbed solution achieves a higher achievable rate than the current global best solution, it replaces the original global best. This mutation-like mechanism enables the swarm to escape local optima while preserving the convergence behavior of the PSO algorithm.

The corresponding implementation is shown below.

```

1 for t in range(max_iter):
2     w = 0.9 - 0.5 * (t / max_iter)
3
4     for i in range(num_particles):
5         r1, r2 = np.random.rand(DIM), np.random.rand(DIM)
6         V[i] = chi * (w * V[i] + phi1 * r1 * (pbest[i] - X[i]) + phi2 * r2 * (gbest - X[i]))
7         X[i] = X[i] + V[i]
8
9         X[i,0::4]=np.clip(X[i,0::4],L1_MIN,L1_MAX)
10
11        X[i,1::4]=np.clip(X[i,1::4],L2_MIN,L2_MAX)
12
13        X[i,2::4]=np.clip(X[i,2::4],C_MIN,C_MAX)
14
15        X[i,3::4]=np.clip(X[i,3::4],R_MIN,R_MAX)
16
17        current_fit = compute_fitness(X[i], Phi, h_d)
18        if current_fit > pbest_fitness[i]:
19            pbest[i] = X[i].copy()
20            pbest_fitness[i] = current_fit
21
22        current_gbest_idx = np.argmax(pbest_fitness)
23        if pbest_fitness[current_gbest_idx] > gbest_fitness:
24            gbest = pbest[current_gbest_idx].copy()
25            gbest_fitness = pbest_fitness[current_gbest_idx]
26
27        sigma = sigma_max - (sigma_max - sigma_min) * (t / max_iter)
28        els_gbest = gbest.copy()
29        d = np.random.randint(0, DIM)
30
31        if d % 4 == 0: # L1
32            els_gbest[d] += (L1_MAX-L1_MIN) * np.random.normal(0, sigma)

```

```

33         els_gbest[d] = np.clip(els_gbest[d], L1_MIN,
34                                 L1_MAX)
35     elif d % 4 == 1:      # L2
36         els_gbest[d] += (L2_MAX-L2_MIN) * np.random.
37                             normal(0, sigma)
38         els_gbest[d] = np.clip(els_gbest[d], L2_MIN,
39                                 L2_MAX)
40     elif d % 4 == 2:      # C
41         els_gbest[d] += (C_MAX-C_MIN) * np.random.normal
42                             (0, sigma)
43         els_gbest[d] = np.clip(els_gbest[d], C_MIN,
44                                 C_MAX)
45     else:                  # R
46         els_gbest[d] += (R_MAX-R_MIN) * np.random.normal
47                             (0, sigma)
48         els_gbest[d] = np.clip(els_gbest[d], R_MIN,
49                                 R_MAX)
50
51     els_fitness = compute_fitness(els_gbest, Phi, h_d)
52     if els_fitness > gbest_fitness:
53         gbest = els_gbest.copy()
54         gbest_fitness = els_fitness
55
56 return gbest

```

Listing 4: Iterative optimization of Adaptive ELS-PSO

3 Flow Chart

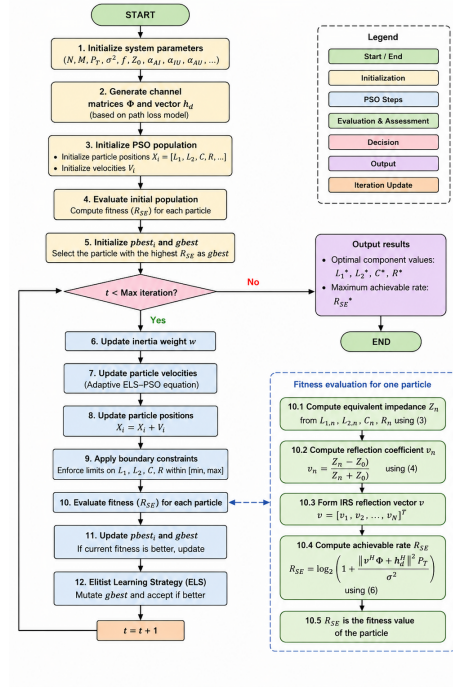


Figure 1: Flow chart of the proposed Adaptive ELS-PSO algorithm.

4 Results

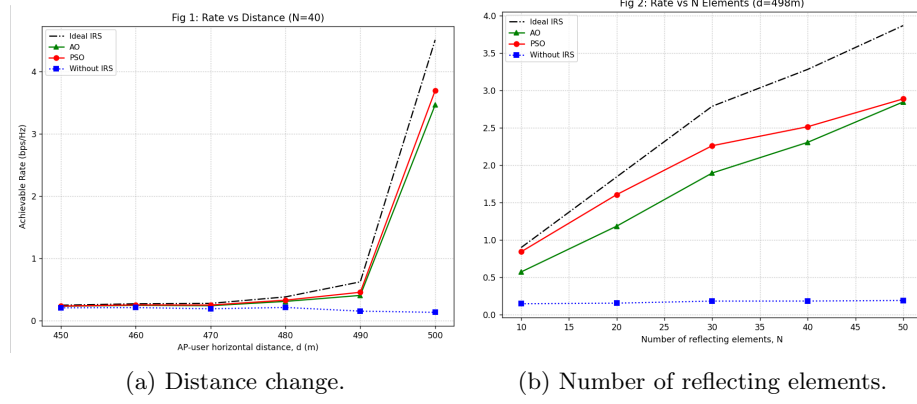


Figure 2: Simulation results obtained using the proposed Adaptive ELS-PSO algorithm.